```
# Install once in Google Colab
!pip install -q qiskit qiskit-aer pylatexenc
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt
# Secret number
secretNumber = "101"
n = len(secretNumber)
# n input qubits + 1 ancilla qubit
# n classical bits for measurement
circuit = QuantumCircuit(n + 1, n)
# --------------------------------
# Step 1: Prepare the ancilla |1>
# --------------------------------
circuit.x(n)
# --------------------------------
# Step 2: Apply Hadamard gates
# --------------------------------
for qubit in range(n + 1):
    circuit.h(qubit)
circuit.barrier()
# --------------------------------
# Step 3: Bernstein-Vazirani Oracle
# --------------------------------
```

```
# Reverse because Qiskit uses little-endian qubit
ordering
for i, bit in enumerate(reversed(secretNumber)):
    if bit == '1':
        circuit.cx(i, n)
circuit.barrier()
# --------------------------------
# Step 4: Apply Hadamard gates
# to input qubits
# --------------------------------
for qubit in range(n):
    circuit.h(qubit)
# --------------------------------
# Step 5: Measure input qubits
# --------------------------------
for qubit in range(n):
    circuit.measure(qubit, qubit)
# Display circuit
display(circuit.draw("mpl"))

# --------------------------------
# Step 6: Run on simulator
# --------------------------------
simulator = AerSimulator()
result = simulator.run(
```

```
    circuit,
    shots=1024
).result()
counts = result.get_counts()
print("Measurement results:", counts)
# Display histogram
plot_histogram(counts)
plt.show()
# Most probable result
answer = max(counts, key=counts.get)
print("Secret number is:", answer)
```

Explanation : **Bernstein–Vazirani algorithm for `s = 101`**, explained step by step.

The purpose of the Bernstein–Vazirani algorithm is to find an unknown binary string, called the **secret string**.

Here,

[s = 101]

So our goal is for the quantum circuit to discover `101`.

1. Import the required libraries

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
```

```
from qiskit.visualization import
plot_histogram
import matplotlib.pyplot as plt
```

These libraries are used for different purposes:

- `QuantumCircuit` → creates the quantum circuit.
- `AerSimulator` → simulates the quantum computer.
- `plot_histogram` → displays the measurement results graphically.
- `matplotlib.pyplot` → helps display the graph.

---

2. Define the secret string

```
secretNumber = "101"
```

This is the hidden binary string that the algorithm has to determine.

[s = 101]

The length of this string is 3.

```
n = len(secretNumber)
```

Therefore:

```
n = 3
```

This means we need **3 input qubits**.

---

3. Create the quantum circuit

```
circuit = QuantumCircuit(n + 1, n)
```

Since:

```
n = 3
```

this becomes:

```
circuit = QuantumCircuit(4, 3)
```

So the circuit contains:

```
3 input qubits
+
1 ancilla qubit
----------------
4 quantum bits
```

and:

```
3 classical bits
```

The circuit can be visualized as:

```
q0 ——— Input qubit
q1 ——— Input qubit
q2 ——— Input qubit
q3 ——— Ancilla qubit

c0 ——— Classical bit
c1 ——— Classical bit
c2 ——— Classical bit
```

The classical bits store the final measurement.

Step 1: Prepare the ancilla qubit

```
circuit.x(n)
```

Since:

```
n = 3
```

this means:

```
circuit.x(3)
```

Initially, all qubits are in state:

[|0 >]

So initially:

[|0000 >]

The last qubit `q3` is the ancilla.

We apply an **X gate** to it.

The X gate changes:

[|0 > → |1 >]

Therefore the ancilla becomes:

[|1 >]

So we now have:

```
q0 = |0>
q1 = |0>
q2 = |0>
q3 = |1>   ← Ancilla
```

The ancilla is prepared in `|1⟩` because it will later be used to create a **phase kickback**.

Step 2: Apply Hadamard gates

```
for qubit in range(n + 1):
    circuit.h(qubit)
```

Since `n + 1 = 4`, the loop applies H gates to:

`q0`

```
q1
q2
q3
```

Equivalent code would be:

```
circuit.h(0)
circuit.h(1)
circuit.h(2)
circuit.h(3)
```

What does the H gate do?

For a qubit in state $|0\rangle$:

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

It creates **superposition**.

Therefore the first three qubits simultaneously represent all possible 3-bit combinations:

[000,001,010,011,100,101,110,111]

This is one of the important quantum ideas used by the algorithm.

What happens to the ancilla?

The ancilla was initially:

[|1 >]

After applying H:

$$H|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

This state allows the oracle to encode information about the secret string into the **phase** of the input qubits.

Step 3: Add a barrier

```
circuit.barrier()
```

A barrier does **not perform any quantum operation**.

It is mainly used to make the circuit diagram easier to read.

You can think of it as separating the circuit into stages:

```
Preparation | Oracle | Measurement
```

Step 4: Construct the Bernstein–Vazirani oracle

```
for i, bit in
enumerate(reversed(secretNumber)):
    if bit == '1':
        circuit.cx(i, n)
```

This is the most important part.

Our secret string is:

```
101
```

We reverse it because of Qiskit's qubit ordering:

```
101
```

In this particular case, reversing `101` still gives:

```
101
```

Now the loop examines each bit.

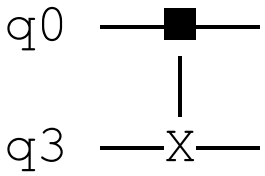
First bit

`bit = 1`

Therefore:

```
circuit.cx(0, 3)
```

A CNOT gate is applied:



where:

- `q0` = control qubit
- `q3` = ancilla/target qubit

Second bit

`bit = 0`

Since the bit is `0`, no CNOT is added.

Third bit

`bit = 1`

Therefore:

```
circuit.cx(2, 3)
```

So for:

[s=101]

the oracle effectively contains:

```
circuit.cx(0, 3)
circuit.cx(2, 3)
```

The middle qubit has no CNOT because the middle secret bit is `0`.

Conceptually:

```
Secret = 1   0   1
          ↓      ↓
        CNOT   CNOT
```

The oracle encodes the secret `101` into the phases of the input qubits.

Step 5: Add another barrier

```
circuit.barrier()
```

Again, this is only for visual separation.

Now the circuit is divided into:

```
State preparation

        ↓

Oracle

        ↓

Final processing
```

Step 6: Apply Hadamard gates again

```
for qubit in range(n):
```

```
    circuit.h(qubit)
```

Notice that this time:

```
range(n)
```

means only:

```
q0
q1
q2
```

The ancilla `q3` is not included.

Equivalent code:

```
circuit.h(0)
circuit.h(1)
circuit.h(2)
```

These Hadamard gates convert the **phase information introduced by the oracle** into ordinary computational-basis information that can be measured.

After this step, the input qubits correspond to:

[|101 >]

That is the central trick of Bernstein–Vazirani.

Step 7: Measure the input qubits

```
for qubit in range(n):
    circuit.measure(qubit, qubit)
```

This becomes:

```
circuit.measure(0, 0)
circuit.measure(1, 1)
```

```
circuit.measure(2, 2)
```

Each quantum bit is measured into its corresponding classical bit.

```
q0 → c0
q1 → c1
q2 → c2
```

The ancilla is **not measured**, because we do not need its result.

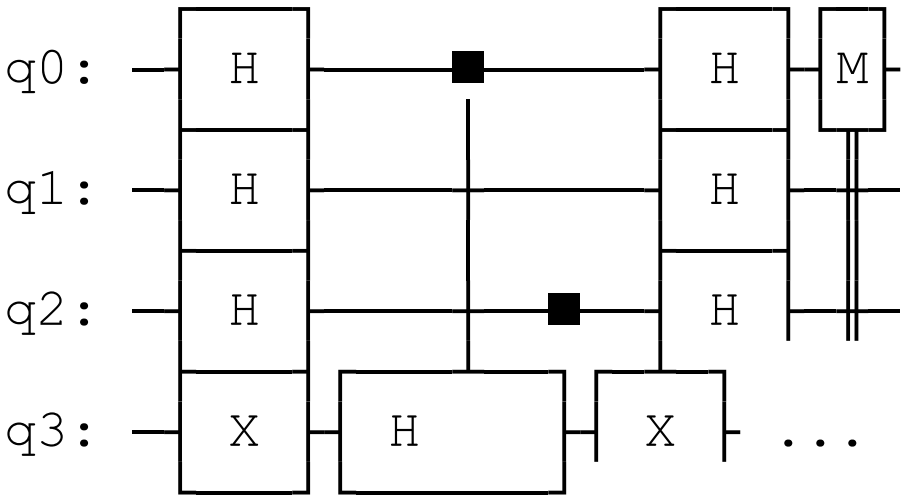
The important information is contained in the first three qubits.

Step 8: Draw the quantum circuit

```
display(circuit.draw("mpl"))
```

This displays the circuit graphically.

You should see a structure approximately like:



The exact drawing may look slightly different depending on your Qiskit version.

Step 9: Create the simulator

```
simulator = AerSimulator()
```

This creates a simulated quantum computer.

Instead of running the circuit on an actual IBM quantum processor, we run it locally using the Aer simulator.

Step 10: Execute the circuit

```
result = simulator.run(
    circuit,
    shots=1024
).result()
```

Here:

```
shots = 1024
```

means the circuit is executed **1024 times**.

The simulator measures the result each time.

Because Bernstein–Vazirani is deterministic on an ideal quantum simulator, we expect:

```
101
```

almost every time—in fact, ideally every time.

Step 11: Obtain the measurement counts

```
counts = result.get_counts()
```

This collects the measurement results.

For example:

```
{'101': 1024}
```

means:

```
101 was measured 1024 times.
```

Step 12: Print the results

```
print("Measurement results:", counts)
```

Output:

```
Measurement results: {'101': 1024}
```

This tells us that the quantum circuit successfully identified the secret string.

Step 13: Plot the histogram

```
plot_histogram(counts)
plt.show()
```

This creates a bar graph.

You should see a large bar corresponding to:

```
101
```

with probability approximately:

[1]

or:

[100%]

Step 14: Find the secret number

```
answer = max(counts, key=counts.get)
```

This finds the measurement result that occurred the maximum number of times.

For example:

```
counts = {'101': 1024}
```

Therefore:

```
answer = 101
```

Finally:

```
print("Secret number is:", answer)
```

Output:

```
Secret number is: 101
```

Complete flow of Bernstein–Vazirani

For `s = 101`, you can remember the algorithm as:

```
Initial state
      ↓
3 input qubits |000>
1 ancilla      |0>
      ↓
X on ancilla
      ↓
Ancilla becomes |1>
      ↓
Hadamard on all qubits
      ↓
Input qubits enter superposition
      ↓
```

```
Oracle for s = 101
      ↓
CNOT q0 → ancilla
No CNOT for q1
CNOT q2 → ancilla
      ↓
Secret encoded as phase
      ↓
Hadamard on input qubits
      ↓
Phase information converted to bits
      ↓
Measure q0, q1, q2
      ↓
101
```

The key idea

A classical method may need to query the hidden function multiple times to determine all bits of `s`.

Bernstein–Vazirani exploits **superposition and phase kickback** so that the entire secret string

[{s=101}]

can be recovered with **one quantum oracle query**.